

LECTURE - 1 (PART 2)

INTRODUCTION TO ALGORITHMS

Md. Shahidul Islam

Assistant Professor, CSE, UAP

Overview

- Asymptotic Notation
- Complexity Analysis

Asymptotic Notation

- The three most commonly used notations are:
 1. Big O notation (O): represents the upper bound
 2. Omega notation (Ω): represents the lower bound
 3. Theta notation (Θ): represents both the upper and lower bounds

O – Notation

★ When we have only an asymptotic upper bound, we use O-notation

★ For a given function $g(n)$, $O(g(n)) =$

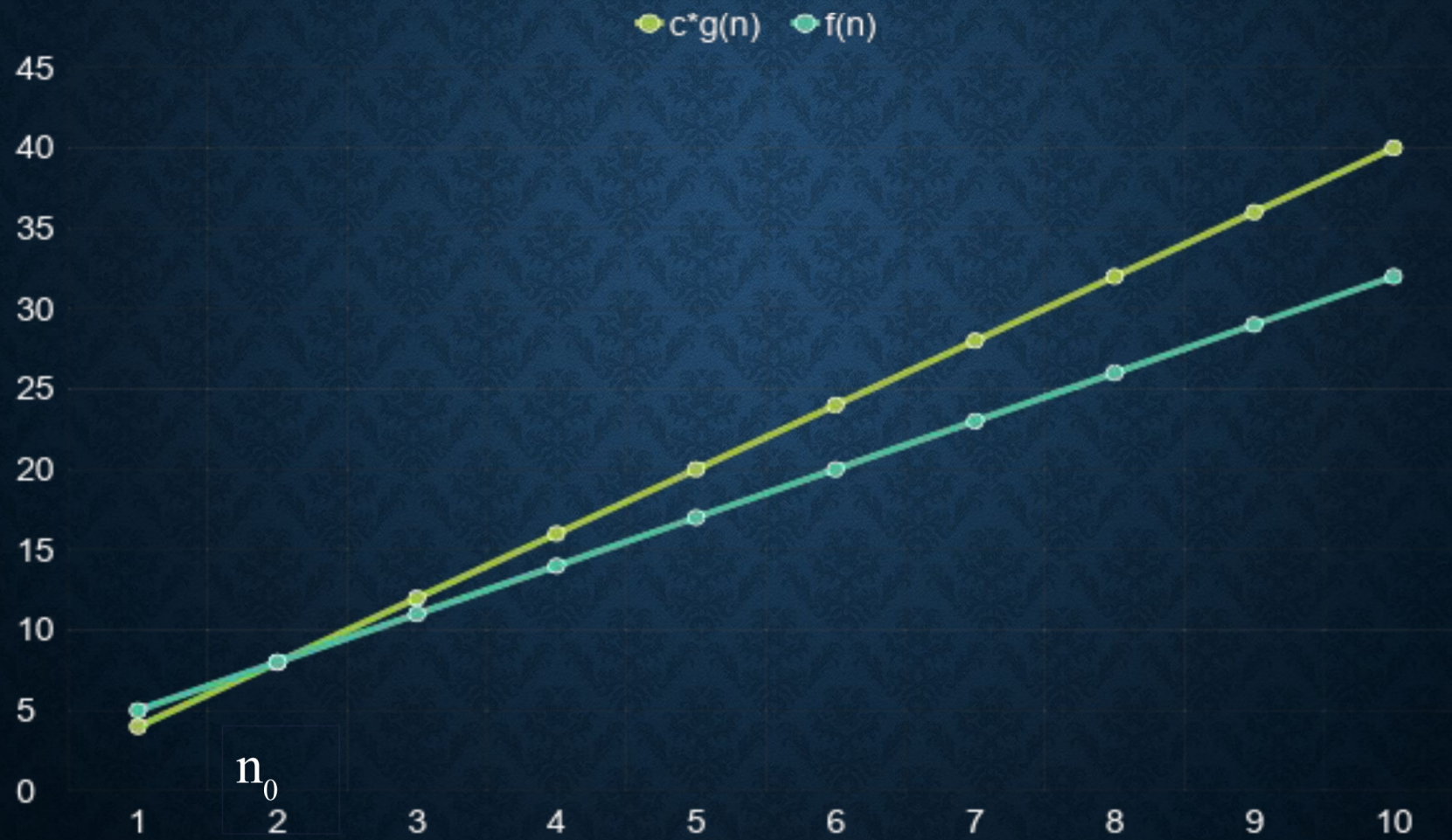
$\{f(n): \text{there exists positive constant } c \text{ and } n_0 \text{ such that}$

$$f(n) \leq c * g(n) \text{ for all } n \geq n_0\}$$

★ For a sufficiently large value of n , the running time of an algorithm does not cross the time provided by $O(g(n))$.

★ Example: $O(n) = 3n+2 \leq 4n$ for all $n \geq 2$.

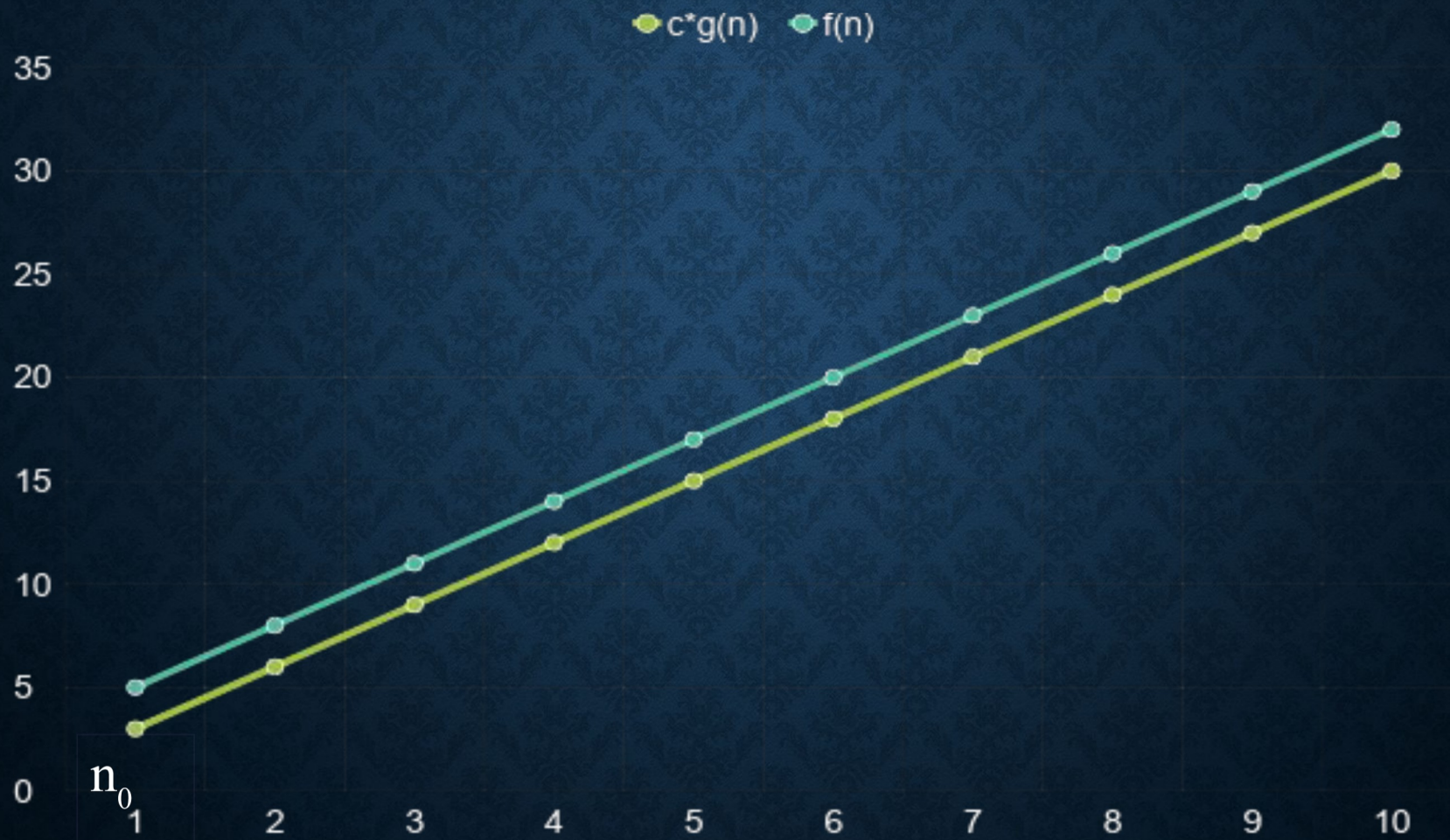
O-notation example



Ω – Notation

- ★ Omega notation represents the lower bound on the growth rate of a function.
- ★ For a given function $g(n)$, $\Omega(g(n)) =$
 $\{f(n): \text{there exists positive constant } c \text{ and } n_0 \text{ such that}$
$$c * g(n) \leq f(n) \text{ for all } n \geq n_0\}$$
- ★ For a sufficiently large value of n , the running time of an algorithm is above the time provided by $\Omega(g(n))$.
- ★ Example: $\Omega(n) = 3n+2 \geq 3n$ for all $n \geq 1$.

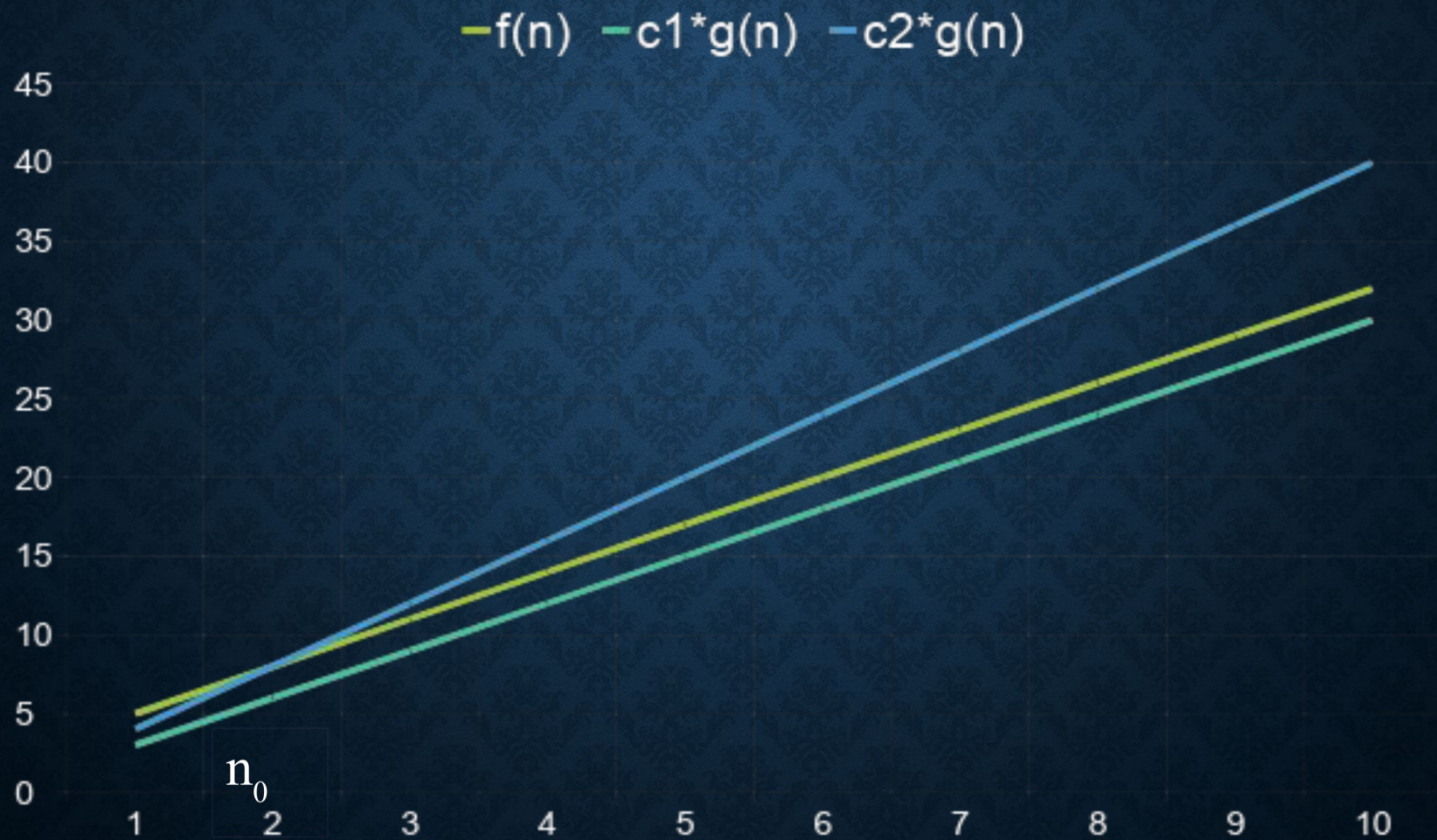
Ω -notation example



Θ – Notation

- ★ Θ represents both the upper and the lower bound on the growth rate of a function.
- ★ For a given function $g(n)$, $\Theta(g(n)) =$
 $\{f(n): \text{there exists positive constant } c_1, c_2 \text{ and } n_0 \text{ such that}$
$$c_1g(n) \leq f(n) \leq c_2g(n) \text{ for all } n \geq n_0\}$$
- ★ For a sufficiently large value of n , the growth rate is neither faster nor slower than the time provided by $\Theta(g(n))$.
- ★ Example: $\Theta(n) = 3n+2 \geq 3n$ for all $n \geq 2$ and $3n + 2 \leq 4n$ for all $n \geq 2$.

Θ -notation example



Complexity Analysis

- Three asymptotic notations can be used to analyze an algorithm in three cases:
 1. Worst Case Analysis:
 - must know the case that causes a maximum number of operations to be executed

Complexity Analysis

2. Best Case Analysis

- must know the case that causes a minimum number of operations to be executed

3. Average Case Analysis

- must know the distribution of cases

Complexity Analysis - Linear Search

- Best Case: element at the first index.
 - $O(1)$
- Average Case: element on the middle index.
 - $O(n/2) \rightarrow O(n)$
- Worst Case: element at the last index or not found
 - $O(n)$

```
1 int search(int arr[], int n, int x)
2 {
3     int i;
4     for (i = 0; i < n; i++)
5     {
6         if (arr[i] == x)
7             return i;
8     }
9     return -1;
10 }
```

Complexity Analysis – More Example



```
int n = 10;  
for(int i=0; i<n; i++)  
{  
    printf("%d", i);  
}
```

$O(n)$

$O(n+m)$



```
int n = 10, m = 20;  
for(int i=0; i<n; i++)  
{  
    printf("%d", i);  
}  
for(int i=0; i<m; i++)  
{  
    printf("%d", i);  
}
```

Complexity Analysis – More Example



```
int n = 10, m=20;
for(int i=0; i<n; i++)
{
    for(int j=0; j<m; j++)
    {
        printf("%d, ", j);
    }
}
```

$O(n*m)$

$O(n*n)$



```
int n = 10;
for(int i=0; i<n; i++)
{
    for(int j=0; j<i; j++)
    {
        printf("%d, ", j);
    }
}
```

O – Notation

- ★ The function $g(n)$ appearing within the $O()$ typically omits constant factors and lower-order terms.
- ★ Let's consider the following functions:
 - $f_1(x) = 3x^2 + 2x + 5$
 - $f_2(x) = 4x^2 + 3x$
 - $f_3(x) = x^2$
- ★ What is the growth pattern of each function?
 - All the above equations are quadratic
 - They have a similar growth

Plot of function values

REFERENCES

THIS PRESENTAION WAS DESIGNED BASED ON:

1. Ellis Horowitz, S. Rajasekaran, Sartaj Sahni: Fundamental of Computer Algorithms, Second Edition, 2008, Universities Press.
2. T.H. Cormen, C.E.Leiserson, R. L. Rivest, C. Stein: Introduction to Algorithms, Third Edition, 2009, PHI Learning Pvt. Ltd.